

PikaPikaGen: Generative Synthesis of Pokémon Sprites from Textual Descriptions

1. Introduction

This report outlines the design, implementation, and evaluation of **PikaPikaGen**, a deep learning model for generating Pokémon sprites from their corresponding Pokédex descriptions. The project tackles the text-to-image synthesis task, which requires a deep semantic understanding of both natural language and visual domains.

The primary objective was to develop a generative model based on an Encoder-Decoder architecture with an integrated Attention mechanism. The model takes a textual description of a Pokémon as input and synthesizes a 2D sprite that accurately reflects the described attributes, such as shape, color, and features.

2. Data Preprocessing

A crucial initial step was to preprocess the suggested dataset to prepare it for training. The dataset is composed of a .csv file containing two columns, the Pokémon national number and its complete description, and a separate folder containing sprite images. To improve the generalization capabilities of the model, we modified the original .csv file which contained incomplete descriptions that didn't focus on the physical characteristics required to generate realistic images. To do so, we used descriptions from another dataset [\[1\]](#), which were more inline with our scope.

2.1. Text Preprocessing

The goal of text preprocessing is to convert the raw, human-readable Pokédex descriptions into a structured, numerical format that the bert-mini encoder can interpret. This involves a sequence of critical operations that handle the complexities of natural language.

2.1.1. Tokenization

First, the raw text is broken down into smaller units called tokens. Instead of simply splitting by spaces, the model uses a WordPiece tokenizer, which is a subword tokenization strategy. This approach has two key advantages:

- **Manages Vocabulary Size:** It breaks down rare or complex words into smaller, more common subword units. For instance, "undeniably" might be tokenized into "un", "##deni", and "##ably". This allows the model to understand a vast vocabulary without needing to store every single word, preventing issues with unknown words.

- **Captures Semantic Relationships:** By breaking words into meaningful parts (like root words and suffixes), the model can better grasp the relationship between related words (e.g., "sleep", "sleeping", "sleepy").

The bert-mini tokenizer also adds special tokens:

- [CLS]: A special token added to the beginning of every sequence. The final hidden state corresponding to this token is often used as an aggregate representation of the entire sequence.
- [SEP]: A token used to indicate the end of a sequence or to separate two different sequences.

2.1.2. Encoding

After tokenization, each token (including the special ones) is mapped to a unique integer ID from the tokenizer's predefined vocabulary. This converts the sequence of text tokens into a sequence of numbers, which is the only format the model can process.

2.1.3. Padding and Truncation

Neural models require inputs to be of a fixed length to perform computations in batches efficiently. Since Pokédex descriptions vary in length, we enforce a uniform size (in this case, 128 tokens):

- **Padding:** If a description is shorter than 128 tokens, special [PAD] tokens (represented by the ID 0) are added to the end until it reaches the required length.
- **Truncation:** If a description is longer than 128 tokens, it is cut short to fit.

This standardization ensures that all input tensors have the same shape, which is essential for batch processing on a GPU.

2.1.4. Creating the Attention Mask

Simply adding padding tokens creates a problem: the model might mistakenly interpret the padding as meaningful parts of the description. The **attention mask** is a binary tensor that solves this by explicitly telling the model which tokens to pay attention to and which to ignore.

- The mask contains a 1 for every real token (including [CLS] and [SEP]).
- It contains a 0 for every [PAD] token.

During the self-attention calculation within the Transformer, this mask ensures that the padding tokens do not contribute to the context vectors. This forces the model to focus its "attention" solely on the actual words of the description, leading to a more accurate and meaningful textual representation [\[2\]](#).

2.2. Image Preprocessing

The sprite images required standardization and augmentation to create a consistent and robust training set:

1. **Loading and Resizing:** All pixel sprite images were loaded and resized to 215x215.
2. **Handling Transparency:** The original images have a transparent background (alpha channel). To create a uniform format, the RGBA images were converted to RGB by blending them with a solid white background.
3. **Normalization:** Pixel values were normalized from the range to $[-1, 1]$. This is a standard practice for training generative models with a Tanh activation function in the final layer, as it helps stabilize training.
4. **Data Augmentation:** For the training set, random horizontal flipping was applied to the images with a probability of 0.5. This technique artificially expands the dataset, helping the model generalize better and preventing overfitting.

2.3. Dataset Splitting

The dataset was divided into three subsets to ensure a rigorous evaluation of the model:

- **Training Set:** 80% of the data, used for training the model.
- **Validation Set:** 10% of the data, used for monitoring performance during training and for hyperparameter tuning.
- **Test Set:** 10% of the data, used for the final, unbiased evaluation of the trained model.

3. Model Architecture

The core of the PikaPikaGen project is an Encoder-Decoder model with an Attention mechanism, as outlined in the project specifications, using mostly PyTorch libraries.

3.1. Text Encoder

The Text Encoder is responsible for processing the input Pokédex descriptions and converting them into a rich, contextualized representation.

- **Embedding Layer:** The input token indices are first mapped to dense vectors using the bert-mini embedding layer.
- **Transformer Encoder:** A pre-trained bert-mini model was used as the text encoder. Its Transformer architecture, with its multi-head self-attention mechanism, produces a sequence of contextual hidden states (256 dimensions each) for every input token. These embeddings capture the semantic meaning of the words in the context of the entire description. The encoder was fine-tuned during training to adapt its representations to the specific vocabulary of the Pokémon domain.

3.2. Image Decoder (Generator)

The Image Decoder, or Generator, is a Convolutional Neural Network (CNN) that synthesizes an image from a latent vector and the encoded text.

- **Input:** The decoder takes as input a 100-dimensional random noise vector (latent vector z), which is concatenated with the text encoder's output for the [CLS] token which was selected since it aggregates global sentence meaning. This provides a starting point for the generation process, with the noise allowing for variations in the output.
- **Transposed Convolutions:** The core of the decoder consists of a series of Transposed Convolution layers. These layers progressively upsample the spatial dimensions of the feature maps, starting from an initial 4x4 representation and expanding to a final 256x256 image, which is then center-cropped to the target 215x215 size.
- **Normalization and Activation:** Each transposed convolution layer is followed by a BatchNorm2d layer to stabilize training and a ReLU activation function to introduce non-linearity. The final layer uses a Tanh activation function to map the output pixel values to the $[-1, 1]$ range.

3.3. Attention Mechanism

An Attention mechanism was integrated to link the text encoder and image decoder, allowing the model to focus on the most relevant words from the description at each stage of image generation.

Step 1: Input Projections

The inputs to the attention module are the decoder's current state (x , the query) and the encoder's full output (y , the source for keys and values). These are first passed through separate linear layers (q_proj , k_proj , v_proj) to project them into the "attention space." This prepares them for the similarity comparison

Step 2: Calculating Attention Scores

To decide which words are most important, the model calculates a similarity score between the query (the current image part) and every key (each word in the description). This is done using a dot product.

The result is a matrix of "attention scores" where a high score means a strong relevance between that part of the image and that word in the description. These scores are then scaled down by the square root of the head dimension (d_head) to stabilize training.

Step 3: Applying Softmax

The raw scores are converted into probabilities using a Softmax function.

The softmax ensures that all attention scores for a single query sum up to 1. This creates a probability distribution, or a set of "attention weights," that represents how much focus the decoder should place on each word in the description. For example, when drawing the tail, the word "fiery" might get a weight of 0.8, while other words get much smaller weights.

Step 4: Computing the Context Vector

The attention weights are then used to create a weighted average of all the Value vectors.

The model multiplies the attention weights by the word embeddings (the values). Words with high weights contribute more to the final output, while irrelevant words contribute almost nothing. The result is a single, blended vector called the context vector. This vector is a highly specialized summary of the input text, tailored specifically to the part of the image being generated at that moment.

Step 5: Injecting the Context into the Decoder

This final context vector is then projected and added back into the decoder's feature map.

By injecting this text-guided information, the attention mechanism directly influences the features the decoder generates next. This process is repeated at multiple stages of the ImageDecoder, allowing the model to refine its focus as it moves from generating coarse outlines to finer details, ensuring the final sprite is a faithful representation of the input description.

4. Training and Evaluation

4.1. Training Process

The model was trained end-to-end on the training dataset using the following configuration:

- **Optimizer:** The AdamW optimizer was used, which is well-suited for training Transformer-based models.
- **Loss Functions:** A composite loss function was employed to balance pixel-level accuracy with perceptual quality:
 1. **L1 Loss (Mean Absolute Error):** This measures the pixel-wise difference between the generated image and the ground-truth image. We preferred it over L2 (MSE) as it tends to produce less blurry images.
 2. **Perceptual Loss:** This loss goes beyond pixel differences and compares the high-level features of the images. It uses a pre-trained VGG19 network to extract feature maps from both the generated and real images and computes a loss based on the difference in these feature spaces. It combines a content loss (ensuring the main objects are similar) and a style loss (ensuring textures and styles are similar) by comparing Gram matrices. The content loss penalizes the model if the high-level "content" representation is different. It

doesn't care about the exact pixel values, only that the features appear in the same location in the feature map for both images. We pass both the generated image and the real image through the VGG19 network. We then stop at a single, relatively deep layer and we extract the feature maps from this layer for both images. Lastly we calculate the L1 difference between these two feature maps. With the style loss we are forcing our model to generate an image where the feature correlations are the same as the real image, effectively replicating its style and texture and producing much sharper results. We pass both images through VGG and extract feature maps from multiple layers and for each feature map, we calculate its Gram matrix. This captures the correlations between different filter responses in a layer. It measures which features tend to appear together, regardless of their location in the image. Lastly, we compute the L1 difference between the Gram matrices of the generated image and the real image [3].

Training Loop: For 300 epochs, the model was trained by feeding it batches of data, calculating the combined loss, and updating the model parameters via backpropagation. Gradient clipping was used to prevent exploding gradients and further stabilize training.

4.2. Evaluation

Model performance was evaluated both quantitatively and qualitatively.

- **Quantitative Metrics:**
 - **L1 Loss:** Lower values indicate better pixel-level reconstruction.
 - **Perceptual Loss:** Lower values suggest better preservation of high-level features and style.
 - **Structural Similarity Index Measure (SSIM):** This metric measures the similarity in structure, luminance, and contrast between two images. A value closer to 1 indicates higher similarity.
- **Qualitative Evaluation:** Throughout training, sample images were generated from a fixed batch of validation data. This allowed for visual inspection of the model's learning progress, checking for improvements in color accuracy, shape coherence, and detail generation. Final results were also generated on the unseen test set.

Training Results (Training Set):

- **L1 Loss:** 0.12
- **Perceptual Loss:** 4
- **SSIM Score:** 0.70

Evaluation Results (Test Set):

- **L1 Loss:** 0.19
- **Perceptual Loss:** 6
- **SSIM Score:** 0.64

A sample of generated images from the training set with its real images



A sample of generated images from the test set with its real images



5. Analysis and Discussion

5.1. Model Performance

The results clearly show the inability of the model to generate complex and realistic images. Though colors and shapes are broadly well represented, the model fails to generate fine-grained details or complex textual description. Abstract concepts are particularly challenging to translate into visual features, which is why the generated images may look simple and blurry. As we can see from the images above, the images generated from the

training set seem to be very similar to the real ones, as one could expect. Pushing training beyond 300 epochs further improves their look but the model quickly runs into overfitting problems which increases the test loss and makes it so that it generates even more abstract images. The image below shows what a generated image from the training set could look like when taking training too far.



5.2. Challenges Faced

1. **Lack of training data:** The dataset provided contained around 900 photos which is not enough for complex image generation tasks.
2. **Overfitting:** To reduce overfitting we tried several strategies, keeping only the ones that produced the best results. Firstly we augmented the dataset using a python script to retrieve additional photos for each Pokémon. This however didn't improve the generalization capabilities suggesting we needed more robust strategies. To do so, we used data augmentation techniques such as random horizontal flipping and weight decay. Dropout and early stopping were also implemented but later dropped as they were too aggressive for this type of model.
3. **Lack of computational power:** We didn't have access to powerful and proper hardware which made it hard for us to train the model due to excessive training time needed. To face this, we used free resources provided by the Google Colab platform.

5.3. Recommendations for Improvement

1. **Advanced Generative Architectures:** Future work could explore more advanced generative models, such as Generative Adversarial Networks (GANs), specifically variants like StyleGAN2. A GAN framework with a discriminator could lead to even more realistic and sharper images.
2. **Conditional Generation:** The model could be enhanced by incorporating additional conditional information, such as the Pokémon's type (e.g., 'Fire', 'Water'). This could be passed to the generator as an additional embedding, providing stronger guidance for color and texture generation.

3. **Hyperparameter Tuning:** Further systematic tuning of hyperparameters, such as the weights of the different loss components, the number of attention heads, and the latent vector dimension, could yield further performance gains.

6. Gradio demo

The Gradio script serves as an interactive web interface for the PikaPikaGen model, making it easy for anyone to use.

Its operation is straightforward and consists of three key parts:

1. **Model Loading:** It first loads the pre-trained Pokémon generator model and its corresponding bert-mini tokenizer. It prepares the model for fast inference by placing it on a GPU (if available) and setting it to evaluation mode.
2. **Inference Function (`generate_pokemon_sprite`):** It takes a text description from the user and performs the following steps:
 - **Tokenizes** the text into a format the model understands.
 - **Feeds** the tokens to the model to generate an image tensor.
 - **Post-processes** the tensor by converting it from a raw PyTorch output into a standard, displayable image.
3. **UI Creation:** It uses the `gr.Interface` function to build the web application. It creates a simple UI with a text box for input and an image component for the output, linking them together with the inference function.

[1] <https://huggingface.co/datasets/jugg1024/pokemon-gpt4o-captions>

[2] <https://arxiv.org/abs/1810.04805>

[3] <https://arxiv.org/abs/1603.08155>

Fabio Dell'Aquila
Jacopo Pagano